03 Password Strength & Breach Checker

03-password-breach-checker

Local zxcvbn/pattern scoring plus k-anonymous Pwned Passwords lookup; only the first five SHA-1 hex characters leave the client.

Verified here: 49 passed; 0 failed; 0 skipped

Measured results

- 49/49 pytest cases passed (48 offline, one live API)
- Live Pwned Passwords test returned HTTP 200; known public password appeared 52,372,427 times
- Only 5 hex characters (5BAA6) sent for public test; full hash and plaintext absent from request
- 25/25 live random-prefix lookups succeeded; median bucket 1,965 known hashes; median latency 67.2 ms
- 200 synthetic passwords per class tested; Word+year+symbol strong rating fell from 65.5% to 0% after pattern cap

Data provenance and caveats

- Have I Been Pwned Pwned Passwords range API https://api.pwnedpasswords.com/range/5BAA6 queried live on 2026-10-27 with public test value.
- 200 synthetic passwords per class from scripts/benchmark_strength.py; 25 random throwaway strings for prefix-size measurement.
- K-anonymity is not cryptographic zero knowledge; server learns the 20-bit hash prefix and client IP/time.
- Plaintext password exists in the local Python process; Streamlit sends it to the local server process.
- Known public password is for demonstration only; no actual user password was requested or bundled.

Project contents

README.md, requirements.txt, cli.py, app.py, src/pwcheck/__init__.py, src/pwcheck/__main__.py, src/pwcheck/assess.py, src/pwcheck/cli.py, src/pwcheck/hibp.py, src/pwcheck/strength.py, tests/conftest.py, tests/test_app.py, tests/test_cli.py, tests/test_hibp.py, tests/test_strength.py, .gitignore, pyproject.toml, samples/anonymity_measurement.json, samples/metrics.json, samples/strength_benchmark.json, scripts/benchmark_strength.py, scripts/live_privacy_demo.py, scripts/measure_anonymity.py

Password Strength & Breach Checker (k-anonymity)

A command-line tool and Streamlit app that scores a password **locally** (zxcvbn pattern matching, character-set entropy and explicit pattern checks) and checks it against the **Have I Been Pwned "Pwned Passwords"** dataset using the **k-anonymity range API**. Only the first 5 hex characters of the password's SHA-1 hash go over the network.

> **About the name "zero-knowledge".** This kind of tool is often called a
> "zero-knowledge" breach checker. That shorthand is **inaccurate**, and this project
> does not claim it. There is **no cryptographic zero-knowledge proof** here.
> What actually happens:
> * **The server learns something.** It sees a 5-character hash prefix (20 bits),
> which narrows the password to one of ~1/1,048,576 of all SHA-1 values. It also
> sees your IP address, the time and the User-Agent. The prefix is shared by about
> 2,000 known breached hashes, so the server cannot tell which of them (if any) you
> typed. That is **k-anonymity**, not zero knowledge.
> * **The plaintext exists on your machine.** The typed password sits in this
> Python process's memory (and, for the web UI, passes from your browser to the
> local Streamlit server). Python strings cannot be reliably wiped, so it can stay in
> memory until the process exits. Malware, a keylogger or a memory dump on your
> machine can see it.
> * **SHA-1 is only the lookup key** of the dataset. It is not a password-storage
> hash. Store passwords with Argon2id, scrypt or bcrypt.
>
> An accurate description: *a k-anonymous breach lookup that never transmits the
> password or its full hash.*

Features

Area	What it does
Local strength	zxcvbn score (0-4), guess estimate, crack-time estimates, zxcvbn warnings
Explainable checks	length, character classes, naive entropy, repeats, alphabetical/numeric sequences, keyboard walks, years, the `Word+digits+symbol` template, common-password and dictionary hits, l33t substitutions
Final score	zxcvbn score, capped by severe findings (common password, PIN, too short, full sequence or keyboard walk ? max 1) and by the `Word2024!` template (? max 2)
Breach check	SHA-1 computed locally; `GET https://api.pwnedpasswords.com/range/<5 hex>`; suffix matched locally; `Add-Padding: true`; descriptive `User-Agent`; timeout; one retry; 429 back-off
Graceful offline	network errors return status `offline`/`error`. Strength analysis still runs and the verdict says the breach status is unknown
No secrets leaked	password not accepted on the command line; never logged; results and JSON hold no plaintext, no hash suffix and no matched password fragments
CLI	`check` (hidden `getpass` prompt), `--stdin`, `--demo-known-password`, `--offline`, `--json`, `explain` (shows what would be sent, sends nothing)
UI	Streamlit app with a password field, a score gauge, a breach result and a "what left this machine?" panel

How it works

```
          your machine                              api.pwnedpasswords.com
 ??????????????????????????????????????????????????????
 ? password ??? zxcvbn + pattern checks       ?  (never leaves the process)
 ?    ?                                ?
```

```
 ?    ??? SHA-1 = 5BAA6 1E4C9B93F3F...68FD8  ?
 ?              ?     ??? suffix (35) ??????? kept local
 ?              ?? prefix (5) ???????????????? GET /range/5BAA6
 ?                                ?   Add-Padding: true
 ?  match suffix locally ?????????????????????????? ~2,000 "SUFFIX:COUNT" lines
 ?  (padding rows with count 0 ignored)     ?    + ~100-150 padding rows
 ??????????????????????????????????????????????????
```

```
03-password-breach-checker/
??? app.py                    Streamlit UI
??? cli.py                    python cli.py ... (no install needed)
??? pyproject.toml            pip install -e .  ? `pwcheck` command
??? requirements.txt
??? src/pwcheck/
?   ??? hibp.py               k-anonymity client, response parser
?   ??? strength.py           zxcvbn + entropy + pattern checks, recommendations
?   ??? assess.py             combines both into a verdict
?   ??? cli.py                argparse CLI (getpass, --stdin, --demo-known-password)
??? scripts/
?   ??? live_privacy_demo.py    captures the exact HTTP request sent to the live API
?   ??? measure_anonymity.py    live bucket-size and latency measurement (random throwaway
 strings)
?   ??? benchmark_strength.py   score distribution on synthetic pattern classes
??? tests/                    pytest: 48 offline tests + 1 opt-in live test
??? samples/                  real outputs from the runs below
??? docs/                     UI screenshots
```

Security design decisions

1. **Prefix only, enforced in code.** `build_url()` raises unless the value is exactly
   5 uppercase hex characters, so a bug cannot send a longer hash.
2. **Local matching.** The 35-character suffix is compared against the response in
   memory and never sent, logged or returned.
3. **Padding.** `Add-Padding: true` makes the API add fake `SUFFIX:0` rows, so response
   size does not reveal the bucket. The parser counts padding rows separately and never
   treats them as hits, including a padding row that happens to equal our suffix (tested).
4. **No password in argv.** Command-line arguments show up in `ps`, shell history and CI
   logs. The CLI uses `getpass` (hidden input) or `--stdin`. The demo flag uses the
   public value `password`, so a demo never needs a secret.
5. **Logging hygiene.** Log lines carry only the prefix (which is already sent to the
   server), status and bucket size. Tests capture logs at DEBUG level and check that the
   password, suffix and full hash are absent.
6. **Fail soft.** Timeouts, DNS failures, 5xx responses and 429 rate limits become
   `offline`/`error` results with a clear message. The CLI exits with code 3 ("breach
   status unknown") rather than crashing or saying "safe".
7. **Bounded work.** Pattern checks look at the first 256 characters. zxcvbn rejects
   input over 72 characters, so it scores the first 72.
8. **UI hygiene.** The Streamlit submit callback moves the value into a one-shot slot
   and blanks the widget. Nothing is cached, written to disk or echoed. Run the UI on
   localhost only.
9. **No real passwords bundled.** The only real breached value used is the public
   test string `password`. Every other example is generated or a made-up phrase.

How to run

```bash
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

```
pip install -r requirements.txt

python cli.py check                       # hidden prompt
python cli.py check --demo-known-password   # public test value "password"
python cli.py check --offline --json        # strength only, JSON output
python cli.py explain                     # show what would be sent, send nothing
echo "my passphrase" | python cli.py check --stdin   # careful: shell history

streamlit run app.py                      # UI at http://localhost:8501  (add ?demo=1 to
 auto-run the demo)

pytest -q                                 # offline test suite
RUN_LIVE=1 pytest -q                      # + the live API test
python scripts/live_privacy_demo.py         # prints the exact request that was sent
python scripts/measure_anonymity.py --n 25
python scripts/benchmark_strength.py
```

Exit codes for `check`: `0` OK, `1` compromised or weak, `2` empty input, `3` breach status unknown (offline/error).

Results (from real runs on 2026-10-27)

Tests: `RUN_LIVE=1 pytest -v` ? **49 passed** (48 offline + 1 live) in 0.77 s (`samples/pytest_output.txt`). The offline tests mock the HTTP session and check, for every request, that:
the URL path ends in **exactly 5 hex characters** equal to the prefix; the plaintext, the 35-character suffix and the full hash appear nowhere in the URL, headers or any request argument; no query string or body is sent; `Add-Padding: true`, the User-Agent and the timeout are set; the parser handles CRLF/LF, lower case, padding and malformed lines; and timeouts, connection errors, 503 and 429 are handled.

Live API, public test value `password` (`samples/live_privacy_demo.txt`):

```
GET https://api.pwnedpasswords.com/range/5BAA6
User-Agent: pwcheck/1.0.0 (MSc portfolio project; k-anonymity client)
Add-Padding: true
(request body: None)
[PASS] path ends in exactly 5 hex chars
[PASS] plaintext absent from request
[PASS] 35-char suffix absent from request
[PASS] full hash absent from request
HTTP 200, 83,294 bytes, 1,978 real suffixes + 145 padding rows for prefix 5BAA6
"count": 52372427   ? seen 52,372,427 times in breaches
```

k-anonymity set size, live (`samples/anonymity_measurement.json`; 25 lookups of random throwaway strings): 25/25 succeeded, 0 found in breaches; real suffixes per bucket **min 1,901 / median 1,965 / max 2,058**; padding rows median 124; latency **median 67 ms**, max 138 ms. A queried prefix hides your hash among roughly 2,000 known breached hashes (and the ~1/1,048,576 slice of all possible SHA-1 values).

Strength benchmark (`samples/strength_benchmark.json`; 200 synthetic passwords per class, final 0-4 score):

Synthetic class	Mean score	% scored 3-4	Mean naive entropy (bits)	Mean zxcvbn log10 guesses
PIN (digits only)	0.99	0.0	19.8	5.4

Keyboard walk + digits	0.99	0.0	35.6	4.9
Word + year + symbol	2.00	0.0	73.1	8.0
l33t word + digits	1.09	0.0	49.7	5.7
Random 8 chars	2.00	0.0	50.0	8.0
Random 12 chars	4.00	100.0	76.8	12.0
Random 16 chars	4.00	100.0	103.2	16.0
5-word passphrase (27-word list)	4.00	100.0	202.7	24.2

Before the `Word+digits+symbol` cap was added, zxcvbn alone rated **65.5 %** of the "Word + year + symbol" class (e.g. `Saffron1999!`) as strong (3-4). After the cap: 0 %.

Key insights

* **Naive entropy misleads.** `Saffron1999!` has 73 "bits" by the charset formula, but zxcvbn estimates ~10^8 guesses (well under a second offline against a fast hash). Pattern-aware estimation matters.
* **Every estimator can be fooled by how the password was generated.** The 5-word passphrases above come from a 27-word list, so their real entropy is only 5 × log2(27) ? **23.8 bits**, yet both estimators rate them "very strong" (202 bits naive, 10^24 zxcvbn). Estimators judge the string, not the process that made it. Real passphrases need a large list (Diceware: 7,776 words ? 12.9 bits per word).
* **A breach hit is the strongest signal.** A password can score 4/4 and still be in breach lists. Attackers try known breached passwords first (credential stuffing), so a "found" result overrides the strength score in the verdict.
* **k-anonymity is a trade-off, not secrecy.** Each lookup reveals 20 bits of the hash. For a single check that is acceptable. Repeated lookups of variants of the same password from one IP reveal more.

Screenshots

`docs/screenshot_demo_result.png` (demo value `password`, compromised) and `docs/screenshot_passphrase_result.png` (a long made-up passphrase, not found, very strong).

Limitations

* zxcvbn's dictionaries are English-centric. Passwords built from words in other languages (for example transliterated Hindi or Telugu) may score higher than they deserve.
* Crack-time figures are zxcvbn's model estimates, not measurements of any real attacker setup.
* Each lookup reveals the 5-character prefix, your IP and the time to the API operator (and to any TLS-intercepting proxy you are behind).
* Python cannot securely erase strings in memory. Use the tool on a machine you trust.
* The Streamlit UI sends the typed value from browser to server over plain HTTP on localhost. Do not deploy it on a shared host for real passwords.
* "Not found" means not in the dataset, not "safe".

Resume bullets

* Built a password security tool in Python that combines zxcvbn pattern analysis and entropy checks with a **k-anonymity** breach lookup against Have I Been Pwned. The tool sends only a **5-character SHA-1 prefix**, verified live (`password` ? 52.4M breach occurrences, 1,978-hash anonymity bucket).
* Wrote **49 pytest tests** with mocked HTTP that assert the exact request contents: a 5-hex-character path, and no plaintext, hash suffix or full hash in the URL, headers, logs or JSON output. Also covered response padding, timeouts, 429 and 5xx handling.
* Measured the privacy guarantee on the live API: anonymity set **median 1,965 hashes per query** (25 lookups) at a **67 ms** median latency, with request padding

  hiding response size.
* Added rule-based score caps that cut false "strong" ratings for the common
  `Word+year+symbol` template from **65.5 % to 0 %** in a 1,600-password synthetic
  benchmark. Shipped a no-argv CLI (getpass/stdin) and a Streamlit UI.

Credits

Breach data: [Have I Been Pwned - Pwned Passwords](https://haveibeenpwned.com/Passwords)
by Troy Hunt (range API, no key required). Strength estimation:
[zxcvbn](https://github.com/dwolfhub/zxcvbn-python) (Python port of Dropbox's zxcvbn).

Password Strength & Breach Checker — ACTUAL SAMPLE OUTPUT

```
LIVE PRIVACY DEMO
=== Exact request sent ===
GET https://api.pwnedpasswords.com/range/5BAA6
User-Agent: pwcheck/1.0.0 (MSc portfolio project; k-anonymity client)
Accept-Encoding: gzip, deflate
Accept: text/plain
Connection: keep-alive
Add-Padding: true
(request body: None)

=== Privacy assertions ===
[PASS] path ends in exactly 5 hex chars
[PASS] plaintext absent from request
[PASS] 35-char suffix absent from request
[PASS] full hash absent from request
[PASS] Add-Padding header present

=== Response (matched locally) ===
HTTP 200, 83,294 bytes, 1,978 real suffixes + 145 padding rows for prefix 5BAA6
{
  "status": "found",
  "pwned": true,
  "count": 52372427,
  "prefix_sent": "5BAA6",
  "bucket_size": 1978,
  "padding_entries": 145,
  "elapsed_ms": 79.5,
  "message": "Seen 52,372,427 times in known breaches."
}

KNOWN PUBLIC PASSWORD CLI DEMO
Demo mode: checking the public test string "password".
Verdict      : COMPROMISED
Strength     : [#----] 0/4 Very weak
Length       : 8 chars, classes: lowercase
Naive entropy: 37.6 bits (assumes random choice; optimistic)
zxcvbn       : ~10^0.5 guesses; offline fast-hash crack: less than a second; online throttled: 2 m
inutes
Breach check : Seen 52,372,427 times in known breaches. (sent prefix 5BAA6 only; 1,978 real suffix
es + 36 padding rows returned)
Findings:
  - 8 characters; 12+ makes guessing much harder.
  - Uses only lowercase; length matters more than symbols, but a short single-class password is we
ak.
  - Contains an entry from a list of commonly used passwords (rank #2 in that list).
zxcvbn warning: This is a top-10 common password.
Recommendations:
  * This password appears in breach data - stop using it everywhere it is used, starting with emai
l and banking.
  * Make it longer: a random 4-5 word passphrase (15+ characters) beats short complex strings.
  * Turn on multi-factor authentication where available; it protects you even if a password leaks.


```

03-password-breach-checker/requirements.txt

```
requests>=2.31
zxcvbn>=4.4.28
streamlit>=1.30
plotly>=5.18
pytest>=7.4

```

03-password-breach-checker/cli.py

```
#!/usr/bin/env python3
"""Convenience entry point: `python cli.py check` (no install needed)."""
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "src"))
from pwcheck.cli import main  # noqa: E402

if __name__ == "__main__":
    sys.exit(main())

```

03-password-breach-checker/app.py

```
"""Streamlit UI:  streamlit run app.py

Privacy notes for this UI:
* the input is a password field; a submit callback moves the value into a
  one-shot slot and blanks the widget, so it is not kept after the check
* the password is never written to disk, logged, cached (no st.cache on it)
  or displayed; only the 5-char hash prefix sent to the API is shown
* Streamlit itself runs as a local web server: the browser sends the typed
  value to this local Python process. Run it on localhost, not on a shared
  server, if you check real passwords.
"""
import os
import sys

import plotly.graph_objects as go
import streamlit as st

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "src"))
from pwcheck import PwnedPasswordsClient, assess  # noqa: E402
from pwcheck.hibp import API_ROOT  # noqa: E402

st.set_page_config(page_title="Password Strength & Breach Checker", page_icon="□", layout="centere
d")
st.title("□ Password Strength & Breach Checker")
st.caption("Strength is scored locally. The breach lookup sends only the first 5 characters "
           "of the password's SHA-1 hash (k-anonymity) to the Pwned Passwords API.")

offline_default = os.environ.get("PWCHECK_OFFLINE", "0") == "1"


def _take_password():
    """Form callback (runs before the script body): move the typed value into a
    one-shot slot and blank the widget, so it is not kept in widget state."""
    st.session_state["_pending_pw"] = st.session_state.get("pw_input", "")
    st.session_state["pw_input"] = ""

with st.form("check", clear_on_submit=True):
    pw = st.text_input("Password", type="password", key="pw_input",
                       help="Never stored, logged or shown. Cleared after each check.")
    breach_on = st.checkbox("Check breach database (k-anonymity range lookup)", value=not offline_
default)
    demo = st.checkbox('Use the public demo value "password" instead', value=False)
    submitted = st.form_submit_button("Check", on_click=_take_password)

# ?demo=1 in the URL runs the public demo value (handy for screenshots/demos)
url_demo = st.query_params.get("demo") == "1"

if submitted or url_demo:
    typed = st.session_state.pop("_pending_pw", "")  # one-shot: removed immediately
    value = "password" if (demo or url_demo) else typed
    if not value:
        st.warning("Enter a password or tick the demo box.")
    else:
        with st.spinner("Analysing locally" + (" and querying range API..." if breach_on else "...
")):
            a = assess(value, client=PwnedPasswordsClient(timeout=5), check_breach=breach_on)
        del value, typed, pw
        s, b = a.strength, a.breach

        color = {"COMPROMISED": "red", "WEAK": "red", "FAIR": "orange"}.get(a.verdict, "green")
        st.markdown(f"### Verdict: :{color}[{a.verdict}]")

        c1, c2, c3 = st.columns(3)
        c1.metric("Strength score", f"{s.score}/4", s.label)
        c2.metric("Naive entropy", f"{s.naive_entropy_bits:.0f} bits")
```

```
        if b.checked:
            c3.metric("Seen in breaches", f"{b.count:,}")
        else:
            c3.metric("Breach check", b.status)

        gauge = go.Figure(go.Indicator(
            mode="gauge+number", value=s.score, title={"text": "Final strength score (0-4)"},
            gauge={"axis": {"range": [0, 4]}, "bar": {"color": color},
                   "steps": [{"range": [0, 1], "color": "#fde0dd"}, {"range": [1, 2], "color": "#fdd0a2"},
                             {"range": [2, 3], "color": "#fff7bc"}, {"range": [3, 4], "color": "#d9f0d3"}]}))
        gauge.update_layout(height=230, margin=dict(l=20, r=20, t=50, b=10))
        st.plotly_chart(gauge, width="stretch")

        if s.crack_time_offline_fast:
            st.write(f"**Estimated crack time** - offline fast hash (10^10 guesses/s): "
                     f"{s.crack_time_offline_fast}; online, throttled: {s.crack_time_online_throttled}")

        if b.checked:
            (st.error if b.pwned else st.success)(b.message)
        else:
            st.info(b.message)

        if s.findings:
            st.subheader("Why")
            for f in s.findings:
                st.write(f"- {f.message}")
        st.subheader("Recommendations")
        for r in a.recommendations:
            st.write(f"- {r}")

        with st.expander("What left this machine?"):
            if b.prefix:
                st.code(f"GET {API_ROOT}{b.prefix}\nUser-Agent: pwcheck/...\nAdd-Padding: true", language="http")
                st.write(f"The API answered with {b.bucket_size:,} real hash suffixes and "
                         f"{b.padding_entries} padding rows for that prefix; the match was done locally.")
            else:
                st.write("Nothing - the breach lookup was skipped.")

with st.expander("How the k-anonymity lookup works (and its limits)"):
    st.markdown(
        "1. SHA-1(password) is computed **in this process**.\n"
        "2. Only the first **5 hex characters** are sent: `GET /range/ABCDE`.\n"
        "3. The API returns every known suffix for that prefix (typically ~800-2,000) plus padding.\n"
        "4. The remaining 35 characters are compared **locally**.\n\n"
        "This is *not* a cryptographic zero-knowledge proof: the server learns the 5-char prefix, "
        "and the plaintext exists in this program's memory while it runs.")

```

03-password-breach-checker/src/pwcheck/__init__.py

```python
"""pwcheck - local password strength scoring + k-anonymity breach lookup.

The password never leaves this process. Only the first 5 hex characters of
its SHA-1 hash are sent to the Pwned Passwords range API.
"""
__version__ = "1.0.0"

from .hibp import BreachResult, PwnedPasswordsClient, parse_range_response, sha1_hex
from .strength import StrengthReport, analyze_strength
from .assess import Assessment, assess

__all__ = [
    "BreachResult", "PwnedPasswordsClient", "parse_range_response", "sha1_hex",
    "StrengthReport", "analyze_strength", "Assessment", "assess", "__version__",
]

```

03-password-breach-checker/src/pwcheck/__main__.py

```
import sys
from .cli import main

sys.exit(main())

```

03-password-breach-checker/src/pwcheck/assess.py

```
"""Combine local strength analysis and the k-anonymity breach lookup into a
single verdict. The Assessment object holds NO plaintext and NO hash."""
from __future__ import annotations

from dataclasses import dataclass

from .hibp import BreachResult, PwnedPasswordsClient
from .strength import StrengthReport, analyze_strength, recommendations


@dataclass
class Assessment:
    strength: StrengthReport
    breach: BreachResult
    recommendations: list[str]

    @property
    def verdict(self) -> str:
        if self.breach.pwned:
            return "COMPROMISED"
        if self.strength.score <= 1:
            return "WEAK"
        if self.strength.score == 2:
            return "FAIR"
        return "STRONG" if self.breach.checked else "STRONG (breach check not completed)"

    def to_dict(self) -> dict:
        return {"verdict": self.verdict, "strength": self.strength.to_dict(),
                "breach": self.breach.to_dict(), "recommendations": self.recommendations}


def assess(password: str, client: PwnedPasswordsClient | None = None,
           check_breach: bool = True) -> Assessment:
    strength = analyze_strength(password)
    if check_breach and password:
        breach = (client or PwnedPasswordsClient()).check(password)
    else:
        breach = BreachResult("skipped", message="Breach lookup skipped (offline mode).")
    recs = recommendations(strength, pwned=breach.pwned if breach.checked else None)
    if breach.status in ("offline", "error"):
        recs.append("Breach status unknown - re-run the check when you are back online.")
    return Assessment(strength, breach, recs)

```

```
"""Command-line interface.

    pwcheck check                      # prompts with getpass (no echo, not in shell history)
    pwcheck check --stdin              # read ONE line from stdin (e.g. piped from a password mana
ger)
    pwcheck check --demo-known-password  # uses the public test string "password"
    pwcheck check --offline            # strength only, no network
    pwcheck check --json               # machine-readable output (never contains the password)
    pwcheck explain                    # show exactly what would be sent for a password, then stop

The password is deliberately NOT accepted as a command-line argument: argv is
visible to other users via `ps`, is saved in shell history, and often ends up
in CI logs.
"""
from __future__ import annotations

import argparse
import getpass
import json
import logging
import sys

from . import __version__
from .assess import assess
from .hibp import API_ROOT, PwnedPasswordsClient, sha1_hex, split_hash

DEMO_PASSWORD = "password"  # public, famously breached test value - not a secret

BAR = {0: "[#----]", 1: "[##---]", 2: "[###--]", 3: "[####-]", 4: "[#####]"}


def _read_password(args) -> str | None:
    if args.demo_known_password:
        return DEMO_PASSWORD
    if args.stdin:
        line = sys.stdin.readline()
        return line.rstrip("\r\n")
    try:
        pw = getpass.getpass("Password to check (input hidden): ")
    except (EOFError, KeyboardInterrupt):
        print("\nCancelled.", file=sys.stderr)
        return None
    return pw


def render_text(a, demo: bool = False) -> str:
    s, b = a.strength, a.breach
    lines = []
    if demo:
        lines.append('Demo mode: checking the public test string "password".')
    lines.append(f"Verdict      : {a.verdict}")
    lines.append(f"Strength     : {BAR[s.score]} {s.score}/4 {s.label}")
    lines.append(f"Length       : {s.length} chars, classes: {', '.join(s.char_classes) or '-'}")
    lines.append(f"Naive entropy: {s.naive_entropy_bits:.1f} bits (assumes random choice; optimist
ic)")
    if s.zxcvbn_guesses_log10 is not None:
        lines.append(f"zxcvbn       : ~10^{s.zxcvbn_guesses_log10:.1f} guesses; offline fast-hash
crack: "
                     f"{s.crack_time_offline_fast}; online throttled: {s.crack_time_online_throttl
ed}")
    if b.status in ("found", "not_found"):
        lines.append(f"Breach check : {b.message} (sent prefix {b.prefix} only; "
                     f"{b.bucket_size:,} real suffixes + {b.padding_entries} padding rows returned
)")
    else:
```

```
        lines.append(f"Breach check : {b.status.upper()} - {b.message}")
    if s.findings:
        lines.append("Findings:")
        lines += [f"  - {f.message}" for f in s.findings]
    if s.zxcvbn_warning:
        lines.append(f"zxcvbn warning: {s.zxcvbn_warning}")
    lines.append("Recommendations:")
    lines += [f"  * {r}" for r in a.recommendations]
    return "\n".join(lines)


def cmd_check(args) -> int:
    pw = _read_password(args)
    if pw is None:
        return 130
    if not pw:
        print("Empty password - nothing to check.", file=sys.stderr)
        return 2
    client = PwnedPasswordsClient(timeout=args.timeout, padding=not args.no_padding)
    a = assess(pw, client=client, check_breach=not args.offline)
    del pw  # drop our reference as early as possible (CPython still may keep copies)
    if args.json:
        print(json.dumps(a.to_dict(), indent=2))
    else:
        print(render_text(a, demo=args.demo_known_password))
    # exit codes useful in scripts: 0 ok, 1 compromised/weak, 3 breach status unknown
    if a.breach.pwned or a.strength.score <= 1:
        return 1
    if a.breach.status in ("offline", "error"):
        return 3
    return 0


def cmd_explain(args) -> int:
    """Show what leaves the machine, without sending anything."""
    pw = _read_password(args)
    if not pw:
        return 2
    prefix, suffix = split_hash(sha1_hex(pw))
    del pw
    print(f"SHA-1 is computed locally: {prefix}{'*' * len(suffix)} (suffix kept on this machine)")
    print(f"Request that would be sent: GET {API_ROOT}{prefix}")
    print("Headers: User-Agent, Add-Padding: true. No password, no suffix, no cookies.")
    print("The server replies with every known suffix for that prefix; matching happens here.")
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="pwcheck", description="Password strength + k-anonymity breac
h checker")
    p.add_argument("--version", action="version", version=f"pwcheck {__version__}")
    p.add_argument("-v", "--verbose", action="store_true", help="log request metadata (prefix only
)")
    sub = p.add_subparsers(dest="cmd", required=True)
    for name, fn, helptext in (("check", cmd_check, "score a password and look it up"),
                               ("explain", cmd_explain, "show what would be sent, send nothing")):
        sp = sub.add_parser(name, help=helptext)
        src = sp.add_mutually_exclusive_group()
        src.add_argument("--stdin", action="store_true", help="read the password from stdin (one l
ine)")
        src.add_argument("--demo-known-password", action="store_true",
                         help='use the public test value "password" instead of prompting')
        sp.set_defaults(func=fn)
        if name == "check":
            sp.add_argument("--offline", action="store_true", help="skip the network lookup")
```

```
            sp.add_argument("--json", action="store_true", help="JSON output")
            sp.add_argument("--timeout", type=float, default=5.0, help="network timeout in seconds
")
            sp.add_argument("--no-padding", action="store_true", help="disable Add-Padding (not re
commended)")
    return p


def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    logging.basicConfig(level=logging.INFO if args.verbose else logging.WARNING,
                        format="%(levelname)s %(name)s: %(message)s", stream=sys.stderr)
    return args.func(args)


if __name__ == "__main__":  # pragma: no cover
    sys.exit(main())

```

```
"""Pwned Passwords k-anonymity range client.

Protocol (https://haveibeenpwned.com/API/v3#PwnedPasswords):

    1. SHA-1 hash the password locally          -> 40 hex chars
    2. send ONLY the first 5 chars (the prefix) -> GET /range/{prefix}
    3. the API returns every known hash SUFFIX (35 chars) that shares that
       prefix, with a breach count: "SUFFIX:COUNT" per line
    4. compare our own suffix against the list LOCALLY

The server therefore learns a 5-hex prefix, which matches roughly a
million possible hashes - but it does learn that prefix. With the
``Add-Padding: true`` header the server also adds fake zero-count suffixes
so the response size does not leak which bucket was asked for.

Security rules this module follows:
    * the plaintext password and the 35-char suffix never appear in a URL,
      header, query string, log line, exception message or repr
    * network failures never raise into the UI - they return a result with
      ``status="offline"`` / ``"error"`` so strength scoring still works
"""
from __future__ import annotations

import hashlib
import logging
import re
import time
from dataclasses import dataclass, field

import requests

from . import __version__

log = logging.getLogger(__name__)

API_ROOT = "https://api.pwnedpasswords.com/range/"
DEFAULT_TIMEOUT = 5.0  # seconds (connect + read, per attempt)
USER_AGENT = f"pwcheck/{__version__} (MSc portfolio project; k-anonymity client)"
PREFIX_LEN = 5
_HEX40 = re.compile(r"^[0-9A-F]{40}$")
_PREFIX_RE = re.compile(r"^[0-9A-F]{5}$")
_LINE_RE = re.compile(r"^([0-9A-F]{35}):(\d+)$")


def sha1_hex(password: str) -> str:
    """Upper-case SHA-1 hex of the UTF-8 password (the format the API uses).

    SHA-1 is used here ONLY because it is the lookup key of the dataset;
    it is not a password-storage hash (use Argon2id/bcrypt for that).
    """
    if not isinstance(password, str):
        raise TypeError("password must be str")
    return hashlib.sha1(password.encode("utf-8")).hexdigest().upper()


def split_hash(full_hash: str) -> tuple[str, str]:
    """Split a 40-char SHA-1 hex into (5-char prefix, 35-char suffix)."""
    full_hash = full_hash.upper()
    if not _HEX40.match(full_hash):
        raise ValueError("expected a 40-character SHA-1 hex digest")
    return full_hash[:PREFIX_LEN], full_hash[PREFIX_LEN:]


@dataclass
class RangeParse:
```

```
    """Parsed body of a /range/{prefix} response."""
    counts: dict[str, int] = field(default_factory=dict)  # suffix -> count (>0)
    padding_entries: int = 0   # zero-count fake rows added by Add-Padding
    malformed_lines: int = 0   # lines that did not match SUFFIX:COUNT

    @property
    def real_entries(self) -> int:
        return len(self.counts)


def parse_range_response(body: str) -> RangeParse:
    """Parse "SUFFIX:COUNT" lines (CRLF or LF). Padding rows have count 0
    and are counted separately and never treated as a breach hit."""
    out = RangeParse()
    for raw in body.splitlines():
        line = raw.strip().upper()
        if not line:
            continue
        m = _LINE_RE.match(line)
        if not m:
            out.malformed_lines += 1
            continue
        suffix, count = m.group(1), int(m.group(2))
        if count == 0:
            out.padding_entries += 1
            continue
        # duplicate suffixes should not happen; keep the max defensively
        out.counts[suffix] = max(count, out.counts.get(suffix, 0))
    return out


@dataclass
class BreachResult:
    """Outcome of one lookup. Contains no plaintext and no hash suffix."""
    status: str                # "found" | "not_found" | "offline" | "error" | "skipped"
    count: int = 0             # times seen in breaches (0 if not found)
    prefix: str = ""           # the ONLY data disclosed to the API
    bucket_size: int = 0       # real suffixes sharing the prefix (anonymity set seen)
    padding_entries: int = 0
    elapsed_ms: float = 0.0
    message: str = ""

    @property
    def pwned(self) -> bool:
        return self.status == "found" and self.count > 0

    @property
    def checked(self) -> bool:
        return self.status in ("found", "not_found")

    def to_dict(self) -> dict:
        return {
            "status": self.status, "pwned": self.pwned, "count": self.count,
            "prefix_sent": self.prefix, "bucket_size": self.bucket_size,
            "padding_entries": self.padding_entries,
            "elapsed_ms": round(self.elapsed_ms, 1), "message": self.message,
        }


class PwnedPasswordsClient:
    """Small, defensive client for the range endpoint."""

    def __init__(self, timeout: float = DEFAULT_TIMEOUT, padding: bool = True,
                 session: requests.Session | None = None, api_root: str = API_ROOT,
                 retries: int = 1):
```

```
        self.timeout = timeout
        self.padding = padding
        self.api_root = api_root
        self.retries = max(0, retries)
        self.session = session or requests.Session()

    def headers(self) -> dict[str, str]:
        h = {"User-Agent": USER_AGENT, "Accept": "text/plain"}
        if self.padding:
            h["Add-Padding"] = "true"
        return h

    def build_url(self, prefix: str) -> str:
        prefix = prefix.upper()
        if not _PREFIX_RE.match(prefix):
            # hard stop: never let anything longer than 5 hex chars out
            raise ValueError("refusing to send anything but a 5-hex-char prefix")
        return self.api_root + prefix

    def fetch_range(self, prefix: str) -> str:
        """GET the range for a 5-char prefix and return the body text.
        Raises requests exceptions; ``check`` turns them into results."""
        url = self.build_url(prefix)
        last_exc: Exception | None = None
        for attempt in range(self.retries + 1):
            try:
                resp = self.session.get(url, headers=self.headers(), timeout=self.timeout)
                if resp.status_code == 429 and attempt < self.retries:
                    # polite backoff; Retry-After is seconds when present
                    wait = min(float(resp.headers.get("Retry-After", "1") or 1), 5.0)
                    time.sleep(wait)
                    continue
                resp.raise_for_status()
                return resp.text
            except (requests.ConnectionError, requests.Timeout) as exc:
                last_exc = exc
                if attempt < self.retries:
                    time.sleep(0.5)
                    continue
                raise
        assert last_exc is not None
        raise last_exc

    def check_hash(self, full_hash: str) -> BreachResult:
        """Look up a full SHA-1 hex (computed locally) by prefix only."""
        prefix, suffix = split_hash(full_hash)
        t0 = time.perf_counter()
        try:
            body = self.fetch_range(prefix)
        except (requests.ConnectionError, requests.Timeout) as exc:
            log.warning("range lookup offline for prefix=%s (%s)", prefix, type(exc).__name__)
            return BreachResult("offline", prefix=prefix,
                                elapsed_ms=(time.perf_counter() - t0) * 1000,
                                message="Breach database unreachable (offline or timed out). "
                                        "Strength analysis still ran locally.")
        except requests.RequestException as exc:
            code = getattr(getattr(exc, "response", None), "status_code", None)
            log.warning("range lookup failed for prefix=%s (HTTP %s)", prefix, code)
            return BreachResult("error", prefix=prefix,
                                elapsed_ms=(time.perf_counter() - t0) * 1000,
                                message=f"Breach API returned an error (HTTP {code}). Try again la
ter.")
        parsed = parse_range_response(body)
        elapsed = (time.perf_counter() - t0) * 1000
        count = parsed.counts.get(suffix, 0)  # LOCAL comparison - suffix never sent
```

```
        status = "found" if count > 0 else "not_found"
        log.info("range lookup prefix=%s status=%s bucket=%d padding=%d",
                 prefix, status, parsed.real_entries, parsed.padding_entries)
        return BreachResult(status, count=count, prefix=prefix,
                            bucket_size=parsed.real_entries,
                            padding_entries=parsed.padding_entries,
                            elapsed_ms=elapsed,
                            message=(f"Seen {count:,} times in known breaches." if count
                                     else "Not found in the Pwned Passwords dataset."))

    def check(self, password: str) -> BreachResult:
        """Hash locally, look up by prefix, match locally."""
        return self.check_hash(sha1_hex(password))

```

03-password-breach-checker/src/pwcheck/strength.py

```
"""Local password strength analysis (no network).

Two independent views are combined:

1. zxcvbn (Dropbox's estimator, Python port) - pattern matching against
   dictionaries, keyboard walks, dates, repeats, l33t substitutions; gives a
   0-4 score and an estimated number of guesses.
2. Our own transparent checks - a naive "charset entropy" (length x
   log2(pool size)) and explicit pattern detectors, so the report can explain
   *why* a password is weak in plain words.

Naive entropy assumes every character was chosen uniformly at random, which
humans never do, so it OVERESTIMATES strength for human-chosen passwords.
That is exactly why we show it next to zxcvbn's pattern-aware estimate.
"""
from __future__ import annotations

import math
import re
import string
from dataclasses import dataclass, field

try:  # zxcvbn is a hard requirement in requirements.txt, but degrade gracefully
    from zxcvbn import zxcvbn as _zxcvbn
except Exception:  # pragma: no cover - exercised only if the package is missing
    _zxcvbn = None

MAX_ANALYZE_LEN = 256  # bound our own pattern checks
ZXCVBN_MAX_LEN = 72    # zxcvbn refuses longer input (and is super-linear); score the first 72 cha
rs

KEYBOARD_ROWS = ["qwertyuiop", "asdfghjkl", "zxcvbnm", "1234567890",
                 "qwertzuiop", "azertyuiop"]
SCORE_LABELS = ["Very weak", "Weak", "Fair", "Strong", "Very strong"]


@dataclass
class Finding:
    code: str
    message: str


@dataclass
class StrengthReport:
    length: int
    charset_size: int
    char_classes: list[str]
    naive_entropy_bits: float
    zxcvbn_score: int | None          # 0..4
    zxcvbn_guesses_log10: float | None
    crack_time_offline_fast: str | None   # 1e10 guesses/s (fast hash, GPU rig)
    crack_time_online_throttled: str | None  # 100 guesses/hour
    findings: list[Finding] = field(default_factory=list)
    zxcvbn_warning: str = ""
    zxcvbn_suggestions: list[str] = field(default_factory=list)

    @property
    def score(self) -> int:
        """Final 0-4 score: zxcvbn when available, else an entropy fallback,
        then capped by any severe pattern finding."""
        if self.zxcvbn_score is not None:
            base = self.zxcvbn_score
        else:
            base = entropy_to_score(self.naive_entropy_bits)
        if any(f.code in SEVERE for f in self.findings):
```

```
            base = min(base, 1)
        elif any(f.code in MODERATE for f in self.findings):
            base = min(base, 2)
        return base

    @property
    def label(self) -> str:
        return SCORE_LABELS[self.score]

    def to_dict(self) -> dict:
        return {
            "length": self.length, "charset_size": self.charset_size,
            "char_classes": self.char_classes,
            "naive_entropy_bits": round(self.naive_entropy_bits, 1),
            "zxcvbn_score": self.zxcvbn_score,
            "zxcvbn_guesses_log10": None if self.zxcvbn_guesses_log10 is None
            else round(self.zxcvbn_guesses_log10, 2),
            "crack_time_offline_fast": self.crack_time_offline_fast,
            "crack_time_online_throttled": self.crack_time_online_throttled,
            "score": self.score, "label": self.label,
            "findings": [f.__dict__ for f in self.findings],
            "zxcvbn_warning": self.zxcvbn_warning,
            "zxcvbn_suggestions": self.zxcvbn_suggestions,
        }


# Findings that cap the final score. "Word+digits+symbol" is capped at 2 because
# attacker mask rules (e.g. hashcat ?u?l?l?l?l?l?d?d?d?d?s) target exactly that shape,
# which zxcvbn alone tends to under-penalise.
MODERATE = {"predictable_structure"}
SEVERE = {"too_short", "common_password", "single_class_digits", "all_same_char", "keyboard_walk_full",
          "sequence_full"}


def entropy_to_score(bits: float) -> int:
    """Fallback mapping when zxcvbn is unavailable (rough NIST-style bands)."""
    if bits < 28:
        return 0
    if bits < 36:
        return 1
    if bits < 60:
        return 2
    if bits < 80:
        return 3
    return 4


def char_classes(pw: str) -> tuple[list[str], int]:
    """Return the character classes present and the implied pool size."""
    classes, pool = [], 0
    if any(c in string.ascii_lowercase for c in pw):
        classes.append("lowercase"); pool += 26
    if any(c in string.ascii_uppercase for c in pw):
        classes.append("uppercase"); pool += 26
    if any(c in string.digits for c in pw):
        classes.append("digits"); pool += 10
    if any(c in string.punctuation or c == " " for c in pw):
        classes.append("symbols"); pool += 33
    if any(ord(c) > 127 for c in pw):
        classes.append("unicode"); pool += 100  # conservative guess
    return classes, pool


def naive_entropy(pw: str) -> float:
```

```
    _, pool = char_classes(pw)
    return len(pw) * math.log2(pool) if pw and pool else 0.0


def _longest_run(pw: str, step: int) -> int:
    """Longest run of consecutive code points with a fixed step (1 = abc/123,
    -1 = cba/321, 0 = aaa)."""
    if not pw:
        return 0
    best = cur = 1
    for a, b in zip(pw, pw[1:]):
        if ord(b) - ord(a) == step:
            cur += 1
            best = max(best, cur)
        else:
            cur = 1
    return best


def _keyboard_run(pw: str) -> int:
    """Longest substring that is a forward/backward walk along a keyboard row."""
    low = pw.lower()
    best = 0
    for row in KEYBOARD_ROWS:
        for seq in (row, row[::-1]):
            for i in range(len(low)):
                j = i
                while j < len(low) and (j - i) < len(seq):
                    sub = low[i:j + 1]
                    if sub in seq:
                        j += 1
                    else:
                        break
                best = max(best, j - i)
    return best


def detect_patterns(pw: str) -> list[Finding]:
    """Explicit, explainable weakness checks."""
    f: list[Finding] = []
    n = len(pw)
    if n == 0:
        return [Finding("empty", "Password is empty.")]
    if n < 8:
        f.append(Finding("too_short", f"Only {n} characters; use at least 12 (15+ is better)."))
    elif n < 12:
        f.append(Finding("short", f"{n} characters; 12+ makes guessing much harder."))
    classes, _ = char_classes(pw)
    if classes == ["digits"]:
        f.append(Finding("single_class_digits", "Digits only (like a PIN) - trivially enumerable."
))
    elif len(classes) == 1 and n < 16:
        f.append(Finding("single_class", f"Uses only {classes[0]}; length matters more than "
                                         "symbols, but a short single-class password is weak."))
    if len(set(pw)) == 1 and n > 1:
        f.append(Finding("all_same_char", "Every character is the same."))
    elif _longest_run(pw, 0) >= 3:
        f.append(Finding("repeat", "Contains a run of the same character (e.g. 'aaa')."))
    seq = max(_longest_run(pw.lower(), 1), _longest_run(pw.lower(), -1))
    if seq >= 4:
        code = "sequence_full" if seq == n else "sequence"
        f.append(Finding(code, f"Contains an alphabetical/numeric sequence of {seq} characters."))
    kb = _keyboard_run(pw)
    if kb >= 4:
        code = "keyboard_walk_full" if kb == n else "keyboard_walk"
```

```
        f.append(Finding(code, f"Contains a keyboard walk of {kb} keys (e.g. 'qwer', 'asdf')."))
    if re.search(r"(19[5-9]\d|20[0-4]\d)", pw):
        f.append(Finding("year", "Contains a year - years and dates are among the first guesses.")
)
    if re.search(r"^[A-Z][a-z]+\d+[!@#$%^&*.?]?$", pw):
        f.append(Finding("predictable_structure",
                         "Follows the common 'Word + digits + symbol' shape attackers try first.")
)
    return f


DICT_NAMES = {
    "passwords": "a list of commonly used passwords",
    "english_wikipedia": "common English words",
    "female_names": "common first names", "male_names": "common first names",
    "surnames": "common surnames", "us_tv_and_film": "words from TV and film",
}


def findings_from_zxcvbn(sequence: list[dict], length: int = 0) -> list[Finding]:
    """Turn zxcvbn's matched patterns into explainable findings.
    We deliberately never copy the matched token (a password fragment)."""
    out: list[Finding] = []
    seen: set[str] = set()
    for m in sequence:
        if m.get("pattern") != "dictionary" or len(m.get("token", "")) < 4:
            continue
        name = m.get("dictionary_name", "")
        if m.get("l33t") and "leet" not in seen:
            seen.add("leet")
            out.append(Finding("leet", "Uses l33t substitutions (@->a, 0->o...) on a dictionary wo
rd; "
                                       "cracking tools undo these automatically."))
        covers = len(m["token"]) / max(length, 1)
        if name == "passwords":
            # severe only when the common password IS (most of) the password
            code = "common_password" if covers >= 0.6 else "contains_common_password"
        else:
            code = "dictionary_word"
        if code != "common_password" and length >= 16:
            continue  # long multi-word passphrases are SUPPOSED to contain words
        if code not in seen and m.get("rank", 10**9) <= 20000:
            seen.add(code)
            where = DICT_NAMES.get(name, "a dictionary")
            rev = " (reversed)" if m.get("reversed") else ""
            out.append(Finding(code, f"Contains an entry from {where}{rev} "
                                     f"(rank #{m.get('rank'):,} in that list)."))
    return out


def recommendations(report: "StrengthReport", pwned: bool | None = None) -> list[str]:
    """Plain-language next steps, most important first."""
    recs: list[str] = []
    if pwned:
        recs.append("This password appears in breach data - stop using it everywhere it is used, "
                    "starting with email and banking.")
    codes = {f.code for f in report.findings}
    if report.length < 15:
        recs.append("Make it longer: a random 4-5 word passphrase (15+ characters) beats short com
plex strings.")
    if codes & {"sequence", "sequence_full", "keyboard_walk", "keyboard_walk_full", "repeat", "all
_same_char"}:
        recs.append("Remove sequences, repeats and keyboard patterns.")
    if codes & {"year", "predictable_structure", "leet"}:
        recs.append("Avoid names, dates and 'Word123!' templates; substitutions like @ for a do no
```

```
t help.")
    if report.score >= 3 and not pwned:
        recs.append("Looks strong. Keep it unique to one site and store it in a password manager."
)
    recs.append("Turn on multi-factor authentication where available; it protects you even if a pa
ssword leaks.")
    # de-duplicate, keep order
    seen, out = set(), []
    for r in recs:
        if r not in seen:
            seen.add(r); out.append(r)
    return out


def analyze_strength(password: str) -> StrengthReport:
    """Run all local checks. Never logs or stores the password."""
    pw = password[:MAX_ANALYZE_LEN]
    classes, pool = char_classes(pw)
    zs = zg = None
    t_fast = t_online = None
    warning, suggestions = "", []
    if _zxcvbn is not None and pw:
        r = _zxcvbn(pw[:ZXCVBN_MAX_LEN])
        zs = int(r["score"])
        zg = float(r.get("guesses_log10") or math.log10(max(float(r["guesses"]), 1)))
        ct = r["crack_times_display"]
        t_fast = ct["offline_fast_hashing_1e10_per_second"]
        t_online = ct["online_throttling_100_per_hour"]
        warning = r["feedback"].get("warning") or ""
        suggestions = list(r["feedback"].get("suggestions") or [])
        extra = findings_from_zxcvbn(r.get("sequence") or [], len(pw))
    else:
        extra = []
    return StrengthReport(
        length=len(password), charset_size=pool, char_classes=classes,
        naive_entropy_bits=naive_entropy(pw), zxcvbn_score=zs,
        zxcvbn_guesses_log10=zg, crack_time_offline_fast=t_fast,
        crack_time_online_throttled=t_online, findings=detect_patterns(pw) + extra,
        zxcvbn_warning=warning, zxcvbn_suggestions=suggestions,
    )

```

03-password-breach-checker/tests/conftest.py

```
import hashlib
import os
import secrets
import sys

import pytest
import requests

sys.path.insert(0, os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "src"
))

# Synthetic test value generated for these tests - not a real password.
TEST_SECRET = "unit-test-value-7f3c-not-a-real-password"


def sha1_upper(s: str) -> str:
    return hashlib.sha1(s.encode()).hexdigest().upper()


class FakeResponse:
    def __init__(self, text="", status=200, headers=None):
        self.text, self.status_code, self.headers = text, status, headers or {}

    def raise_for_status(self):
        if self.status_code >= 400:
            err = requests.HTTPError(f"{self.status_code}")
            err.response = self
            raise err


class FakeSession:
    """Records every outgoing request instead of touching the network."""

    def __init__(self, responses=None, exc=None):
        self.calls = []
        self.responses = list(responses or [])
        self.exc = exc

    def get(self, url, **kwargs):
        self.calls.append({"url": url, **kwargs})
        if self.exc is not None:
            raise self.exc
        return self.responses.pop(0) if self.responses else FakeResponse("")


def make_range_body(hit_suffix=None, hit_count=0, n_other=600, n_padding=50, crlf=True, seed=1):
    """Build a realistic /range body: random 35-hex suffixes, optional hit, padding rows."""
    rng = secrets.SystemRandom() if seed is None else __import__("random").Random(seed)
    rows = [f"{''.join(rng.choice('0123456789ABCDEF') for _ in range(35))}:{rng.randint(1, 5000)}"
            for _ in range(n_other)]
    rows += [f"{''.join(rng.choice('0123456789ABCDEF') for _ in range(35))}:0" for _ in range(n_pa
dding)]
    if hit_suffix:
        rows.append(f"{hit_suffix}:{hit_count}")
    rng.shuffle(rows)
    return ("\r\n" if crlf else "\n").join(rows)


@pytest.fixture
def secret_parts():
    h = sha1_upper(TEST_SECRET)
    return {"password": TEST_SECRET, "hash": h, "prefix": h[:5], "suffix": h[5:]}

```

03-password-breach-checker/tests/test_app.py

```
import os

from streamlit.testing.v1 import AppTest

APP = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "app.py")


def test_app_loads_without_exceptions(monkeypatch):
    monkeypatch.setenv("PWCHECK_OFFLINE", "1")
    at = AppTest.from_file(APP, default_timeout=60).run()
    assert not at.exception


def test_app_check_offline_does_not_echo_password(monkeypatch):
    monkeypatch.setenv("PWCHECK_OFFLINE", "1")
    pw = "Monsoon2019!"  # synthetic
    at = AppTest.from_file(APP, default_timeout=60).run()
    at.text_input(key="pw_input").input(pw)
    at.button[0].click().run()
    assert not at.exception
    rendered = " ".join(str(m.value) for m in at.markdown) + " ".join(str(m.value) for m in at.met
ric)
    assert "Verdict" in rendered
    assert pw not in rendered
    assert at.session_state["pw_input"] == ""          # widget blanked by the callback
    assert "_pending_pw" not in at.session_state       # one-shot slot consumed

```

03-password-breach-checker/tests/test_cli.py

```
import io
import json

import pytest

from conftest import FakeResponse, FakeSession, make_range_body, sha1_upper
from pwcheck import cli


@pytest.fixture
def fake_net(monkeypatch):
    """Route the CLI's client to a FakeSession; returns the session for inspection."""
    holder = {}

    def factory():
        h = sha1_upper("password")
        body = make_range_body(hit_suffix=h[5:], hit_count=123456)
        holder["s"] = FakeSession([FakeResponse(body)])
        return holder["s"]

    monkeypatch.setattr("pwcheck.hibp.requests.Session", factory)
    return holder


def test_demo_known_password(fake_net, capsys):
    rc = cli.main(["check", "--demo-known-password"])
    out = capsys.readouterr().out
    assert rc == 1
    assert "COMPROMISED" in out and "123,456" in out and "5BAA6" in out
    call = fake_net["s"].calls[0]
    assert call["url"].endswith("/range/5BAA6")
    assert sha1_upper("password")[5:] not in repr(call)


def test_getpass_is_used(monkeypatch, capsys, fake_net):
    monkeypatch.setattr("getpass.getpass", lambda prompt="": "password")
    rc = cli.main(["check"])
    assert rc == 1 and "COMPROMISED" in capsys.readouterr().out


def test_stdin_json_has_no_password(monkeypatch, capsys):
    pw = "velvet tractor monsoon pickle"  # synthetic
    monkeypatch.setattr("sys.stdin", io.StringIO(pw + "\n"))
    rc = cli.main(["check", "--stdin", "--offline", "--json"])
    out = capsys.readouterr().out
    data = json.loads(out)
    assert rc == 0 and data["breach"]["status"] == "skipped"
    assert pw not in out and "tractor" not in out


def test_offline_network_exit_code(monkeypatch, capsys):
    import requests
    monkeypatch.setattr("pwcheck.hibp.requests.Session", lambda: FakeSession(exc=requests.Connecti
onError()))
    monkeypatch.setattr("pwcheck.hibp.time.sleep", lambda s: None)
    monkeypatch.setattr("sys.stdin", io.StringIO("Xk9#mQ2!vL7@pR4z\n"))
    rc = cli.main(["check", "--stdin"])
    out = capsys.readouterr().out
    assert rc == 3 and "OFFLINE" in out and "Strength" in out


def test_password_not_accepted_as_argument():
    with pytest.raises(SystemExit):
        cli.main(["check", "hunter2-positional"])

```

```
65
66 def test_explain_sends_nothing(monkeypatch, capsys):
67     def boom():
68         raise AssertionError("explain must not create a network session")
69     monkeypatch.setattr("pwcheck.hibp.requests.Session", boom)
70     rc = cli.main(["explain", "--demo-known-password"])
71     out = capsys.readouterr().out
72     assert rc == 0 and "GET https://api.pwnedpasswords.com/range/5BAA6" in out
73     assert sha1_upper("password")[5:] not in out
74
75
76 def test_verbose_logs_prefix_only(fake_net, capsys, caplog):
77     import logging
78     caplog.set_level(logging.DEBUG)
79     cli.main(["-v", "check", "--demo-known-password"])
80     err = caplog.text + capsys.readouterr().err
81     assert "prefix=5BAA6" in err
82     assert sha1_upper("password")[5:] not in err and "password" not in err.lower().replace("pwchec
   k", "")
83
```

03-password-breach-checker/tests/test_hibp.py

```
import logging
import os
import re

import pytest
import requests

from conftest import FakeResponse, FakeSession, make_range_body
from pwcheck.hibp import (API_ROOT, PwnedPasswordsClient, parse_range_response,
                          sha1_hex, split_hash)


# ---------- hashing ----------------------------------------------------------
def test_sha1_known_vector():
    # public test vector: SHA-1("password")
    assert sha1_hex("password") == "5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8"


def test_sha1_unicode_is_utf8():
    assert len(sha1_hex("pässwörd-ü")) == 40


def test_split_hash():
    p, s = split_hash("5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8")
    assert p == "5BAA6" and len(s) == 35 and s.startswith("1E4C9")


@pytest.mark.parametrize("bad", ["", "XYZ", "5BAA6", "G" * 40, "A" * 41])
def test_split_hash_rejects_bad(bad):
    with pytest.raises(ValueError):
        split_hash(bad)


# ---------- response parser --------------------------------------------------
def test_parser_crlf_padding_malformed_lowercase():
    body = ("0018A45C4D1DEF81644B54AB7F969B88D65:1\r\n"
            "00d4f6e8fa6eecad2a3aa415eec418d38ec:2\r\n"     # lower-case is normalised
            "011053FD0102E94D6AE2F8B83D76FAF94F6:0\r\n"     # padding row
            "garbage line\r\n"
            "\r\n"
            "012A7CA357541F0AC487871FEEC1891C49C:3")
    r = parse_range_response(body)
    assert r.real_entries == 3
    assert r.padding_entries == 1
    assert r.malformed_lines == 1
    assert r.counts["00D4F6E8FA6EECAD2A3AA415EEC418D38EC"] == 2


def test_parser_lf_and_counts():
    body = make_range_body(n_other=100, n_padding=10, crlf=False)
    r = parse_range_response(body)
    assert r.real_entries == 100 and r.padding_entries == 10 and r.malformed_lines == 0


def test_parser_rejects_wrong_length_suffix():
    r = parse_range_response("ABC:5\n" + "A" * 36 + ":5")
    assert r.real_entries == 0 and r.malformed_lines == 2


# ---------- request privacy --------------------------------------------------
def _assert_private(call, parts):
    url = call["url"]
    assert url.startswith(API_ROOT)
    last = url.rsplit("/", 1)[1]
    assert re.fullmatch(r"[0-9A-F]{5}", last), "path must end in exactly 5 hex chars"
```

```
    assert len(last) == 5
    assert last == parts["prefix"]
    blob = repr(call)  # url + headers + every kwarg
    assert parts["password"] not in blob
    assert parts["suffix"] not in blob
    assert parts["hash"] not in blob
    assert "params" not in call and "data" not in call and "json" not in call


def test_request_sends_only_prefix(secret_parts):
    sess = FakeSession([FakeResponse(make_range_body())])
    PwnedPasswordsClient(session=sess).check(secret_parts["password"])
    assert len(sess.calls) == 1
    _assert_private(sess.calls[0], secret_parts)


def test_headers_padding_user_agent_timeout(secret_parts):
    sess = FakeSession([FakeResponse(make_range_body())])
    PwnedPasswordsClient(session=sess, timeout=3.5).check(secret_parts["password"])
    call = sess.calls[0]
    assert call["headers"]["Add-Padding"] == "true"
    assert call["headers"]["User-Agent"].startswith("pwcheck/")
    assert call["timeout"] == 3.5


def test_padding_can_be_disabled(secret_parts):
    sess = FakeSession([FakeResponse(make_range_body())])
    PwnedPasswordsClient(session=sess, padding=False).check(secret_parts["password"])
    assert "Add-Padding" not in sess.calls[0]["headers"]


@pytest.mark.parametrize("bad", ["5BAA61", "5BAA", "5baa!", "5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8"])
def test_build_url_refuses_anything_but_5_hex(bad):
    with pytest.raises(ValueError):
        PwnedPasswordsClient(session=FakeSession()).build_url(bad)


# ---------- matching ----------------------------------------------------------
def test_found_with_count(secret_parts):
    body = make_range_body(hit_suffix=secret_parts["suffix"], hit_count=4242)
    res = PwnedPasswordsClient(session=FakeSession([FakeResponse(body)])).check(secret_parts["password"])
    assert res.status == "found" and res.pwned and res.count == 4242
    assert res.bucket_size == 601 and res.padding_entries == 50


def test_not_found(secret_parts):
    res = PwnedPasswordsClient(session=FakeSession([FakeResponse(make_range_body())])).check(secret_parts["password"])
    assert res.status == "not_found" and not res.pwned and res.count == 0


def test_padding_row_matching_suffix_is_not_a_hit(secret_parts):
    body = make_range_body() + f"\r\n{secret_parts['suffix']}:0"
    res = PwnedPasswordsClient(session=FakeSession([FakeResponse(body)])).check(secret_parts["password"])
    assert res.status == "not_found"


# ---------- failure handling --------------------------------------------------
@pytest.mark.parametrize("exc", [requests.Timeout("t"), requests.ConnectionError("no route")])
def test_offline_is_graceful(secret_parts, exc, monkeypatch):
    monkeypatch.setattr("pwcheck.hibp.time.sleep", lambda s: None)
    sess = FakeSession(exc=exc)
```

```
    res = PwnedPasswordsClient(session=sess, retries=1).check(secret_parts["password"])
    assert res.status == "offline" and not res.checked and not res.pwned
    assert len(sess.calls) == 2  # one retry
    assert secret_parts["suffix"] not in res.message


def test_http_error_is_graceful(secret_parts):
    sess = FakeSession([FakeResponse("", 503)])
    res = PwnedPasswordsClient(session=sess, retries=0).check(secret_parts["password"])
    assert res.status == "error" and "503" in res.message


def test_429_then_success(secret_parts, monkeypatch):
    monkeypatch.setattr("pwcheck.hibp.time.sleep", lambda s: None)
    body = make_range_body(hit_suffix=secret_parts["suffix"], hit_count=7)
    sess = FakeSession([FakeResponse("", 429, {"Retry-After": "1"}), FakeResponse(body)])
    res = PwnedPasswordsClient(session=sess, retries=1).check(secret_parts["password"])
    assert res.count == 7 and len(sess.calls) == 2
    for c in sess.calls:
        _assert_private(c, secret_parts)


# ---------- no secrets in logs or results ------------------------------------------
def test_no_secret_in_logs_or_result(secret_parts, caplog, monkeypatch):
    caplog.set_level(logging.DEBUG)
    monkeypatch.setattr("pwcheck.hibp.time.sleep", lambda s: None)
    body = make_range_body(hit_suffix=secret_parts["suffix"], hit_count=3)
    for sess in (FakeSession([FakeResponse(body)]), FakeSession(exc=requests.Timeout()),
                 FakeSession([FakeResponse("", 500)])):
        res = PwnedPasswordsClient(session=sess, retries=0).check(secret_parts["password"])
        dump = repr(res) + str(res.to_dict())
        for s in (secret_parts["password"], secret_parts["suffix"], secret_parts["hash"]):
            assert s not in dump
    text = caplog.text
    assert secret_parts["prefix"] in text  # the prefix (already public) is fine to log
    for s in (secret_parts["password"], secret_parts["suffix"], secret_parts["hash"]):
        assert s not in text


# ---------- live API (opt-in) ------------------------------------------------------
@pytest.mark.live
@pytest.mark.skipif(os.environ.get("RUN_LIVE") != "1", reason="set RUN_LIVE=1 to hit the real API"
)
def test_live_known_compromised_password():
    res = PwnedPasswordsClient(timeout=10).check("password")  # public test value
    assert res.status == "found"
    assert res.count > 1_000_000
    assert res.prefix == "5BAA6"
    assert res.bucket_size > 100

```

03-password-breach-checker/tests/test_strength.py

```python
import math

from pwcheck.assess import assess
from pwcheck.strength import (MAX_ANALYZE_LEN, analyze_strength, char_classes,
                              detect_patterns, naive_entropy)


def codes(pw):
    return {f.code for f in analyze_strength(pw).findings}


def test_char_classes_and_entropy():
    classes, pool = char_classes("aB3$")
    assert classes == ["lowercase", "uppercase", "digits", "symbols"] and pool == 95
    assert math.isclose(naive_entropy("aB3$"), 4 * math.log2(95))
    assert naive_entropy("") == 0.0


def test_common_password_is_very_weak():
    r = analyze_strength("password")
    assert r.score == 0 and "common_password" in codes("password")


def test_pin_and_short():
    c = codes("1234")
    assert {"too_short", "single_class_digits", "sequence_full"} <= c
    assert analyze_strength("1234").score == 0


def test_keyboard_walk_repeat_year_structure():
    assert "keyboard_walk" in codes("zxqwertyz9")
    assert "repeat" in codes("Abcxxx91kd!")
    assert {"year", "predictable_structure"} <= codes("Monsoon2019!")


def test_leet_detected_on_dictionary_word():
    assert "leet" in codes("P@ssw0rd!")


def test_random_and_passphrase_are_strong():
    # synthetic strings created for this test
    assert analyze_strength("Xk9#mQ2!vL7@pR4z").score == 4
    r = analyze_strength("velvet tractor monsoon pickle")
    assert r.score == 4 and "dictionary_word" not in {f.code for f in r.findings}


def test_score_capped_by_severe_finding():
    # zxcvbn might score a long all-digit string decently; our rule caps it
    assert analyze_strength("8" * 30).score <= 1


def test_long_input_is_bounded():
    r = analyze_strength("a1B!" * 1000)
    assert r.length == 4000
    assert r.naive_entropy_bits <= MAX_ANALYZE_LEN * math.log2(95) + 1e-6


def test_empty():
    assert {f.code for f in detect_patterns("")} == {"empty"}


def test_report_never_contains_password():
    pw = "Monsoon2019!"
    a = assess(pw, check_breach=False)
    assert pw not in str(a.to_dict())
```

```
    assert "Monsoon" not in str(a.to_dict())  # no matched tokens copied either


def test_recommendations_breach_first():
    from pwcheck.strength import recommendations
    r = analyze_strength("Xk9#mQ2!vL7@pR4z")
    recs = recommendations(r, pwned=True)
    assert recs[0].startswith("This password appears in breach data")
    recs2 = recommendations(r, pwned=False)
    assert any("Looks strong" in x for x in recs2)


def test_assess_offline_skips_network():
    a = assess("Xk9#mQ2!vL7@pR4z", check_breach=False)
    assert a.breach.status == "skipped"
    assert a.verdict.startswith("STRONG")


def test_word_year_symbol_capped_at_fair():
    assert analyze_strength("Saffron1999!").score <= 2

```

03-password-breach-checker/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.egg-info/
build/
dist/
.streamlit/secrets.toml

```

03-password-breach-checker/pyproject.toml

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "pwcheck"
version = "1.0.0"
description = "Local password strength scoring + Pwned Passwords k-anonymity breach lookup"
requires-python = ">=3.9"
dependencies = ["requests>=2.31", "zxcvbn>=4.4.28"]

[project.optional-dependencies]
ui = ["streamlit>=1.30", "plotly>=5.18"]
dev = ["pytest>=7.4"]

[project.scripts]
pwcheck = "pwcheck.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
markers = ["live: hits the real Pwned Passwords API (run with RUN_LIVE=1)"]

```

```
{
  "summary": {
    "lookups": 25,
    "succeeded": 25,
    "found_in_breaches": 0,
    "bucket_size_min": 1901,
    "bucket_size_median": 1965,
    "bucket_size_max": 2058,
    "padding_rows_median": 124,
    "latency_ms_median": 67.2,
    "latency_ms_max": 138.3,
    "measured_at_utc": "2026-10-27T08:15:19Z"
  },
  "lookups": [
    {
      "status": "not_found",
      "bucket_size": 2058,
      "padding_entries": 158,
      "elapsed_ms": 98.7
    },
    {
      "status": "not_found",
      "bucket_size": 1970,
      "padding_entries": 168,
      "elapsed_ms": 67.2
    },
    {
      "status": "not_found",
      "bucket_size": 1924,
      "padding_entries": 91,
      "elapsed_ms": 90.3
    },
    {
      "status": "not_found",
      "bucket_size": 1951,
      "padding_entries": 155,
      "elapsed_ms": 85.1
    },
    {
      "status": "not_found",
      "bucket_size": 1927,
      "padding_entries": 73,
      "elapsed_ms": 49.8
    },
    {
      "status": "not_found",
      "bucket_size": 1971,
      "padding_entries": 148,
      "elapsed_ms": 61.0
    },
    {
      "status": "not_found",
      "bucket_size": 1975,
      "padding_entries": 163,
      "elapsed_ms": 105.9
    },
    {
      "status": "not_found",
      "bucket_size": 1954,
      "padding_entries": 124,
      "elapsed_ms": 47.6
    },
    {
      "status": "not_found",
      "bucket_size": 1934,
```

```
      "padding_entries": 94,
      "elapsed_ms": 98.9
    },
    {
      "status": "not_found",
      "bucket_size": 1961,
      "padding_entries": 66,
      "elapsed_ms": 44.0
    },
    {
      "status": "not_found",
      "bucket_size": 1984,
      "padding_entries": 37,
      "elapsed_ms": 66.6
    },
    {
      "status": "not_found",
      "bucket_size": 1960,
      "padding_entries": 99,
      "elapsed_ms": 35.6
    },
    {
      "status": "not_found",
      "bucket_size": 1965,
      "padding_entries": 143,
      "elapsed_ms": 55.8
    },
    {
      "status": "not_found",
      "bucket_size": 1911,
      "padding_entries": 24,
      "elapsed_ms": 64.3
    },
    {
      "status": "not_found",
      "bucket_size": 2031,
      "padding_entries": 101,
      "elapsed_ms": 91.3
    },
    {
      "status": "not_found",
      "bucket_size": 1928,
      "padding_entries": 194,
      "elapsed_ms": 101.6
    },
    {
      "status": "not_found",
      "bucket_size": 2028,
      "padding_entries": 164,
      "elapsed_ms": 82.1
    },
    {
      "status": "not_found",
      "bucket_size": 1901,
      "padding_entries": 76,
      "elapsed_ms": 67.8
    },
    {
      "status": "not_found",
      "bucket_size": 1973,
      "padding_entries": 83,
      "elapsed_ms": 93.2
    },
    {
      "status": "not_found",
```

```
      "bucket_size": 1976,
      "padding_entries": 98,
      "elapsed_ms": 101.0
    },
    {
      "status": "not_found",
      "bucket_size": 2040,
      "padding_entries": 135,
      "elapsed_ms": 66.7
    },
    {
      "status": "not_found",
      "bucket_size": 1976,
      "padding_entries": 152,
      "elapsed_ms": 138.3
    },
    {
      "status": "not_found",
      "bucket_size": 1902,
      "padding_entries": 104,
      "elapsed_ms": 53.8
    },
    {
      "status": "not_found",
      "bucket_size": 1927,
      "padding_entries": 160,
      "elapsed_ms": 50.2
    },
    {
      "status": "not_found",
      "bucket_size": 2014,
      "padding_entries": 184,
      "elapsed_ms": 48.2
    }
  ]
}
```

```
{
  "tests": {
    "passed": 49,
    "offline": 48,
    "live": 1
  },
  "live_password_demo": {
    "prefix_sent": "5BAA6",
    "count": 52372427,
    "bucket_real_suffixes": 1978,
    "padding_rows": 145,
    "response_bytes": 83294,
    "http": 200
  },
  "anonymity_measurement": {
    "lookups": 25,
    "succeeded": 25,
    "found_in_breaches": 0,
    "bucket_size_min": 1901,
    "bucket_size_median": 1965,
    "bucket_size_max": 2058,
    "padding_rows_median": 124,
    "latency_ms_median": 67.2,
    "latency_ms_max": 138.3,
    "measured_at_utc": "2026-10-27T08:15:19Z"
  },
  "strength_benchmark_mean_score": {
    "PIN (digits only)": 0.99,
    "Keyboard walk + digits": 0.99,
    "Word + year + symbol": 2.0,
    "l33t word + digits": 1.09,
    "Random 8 chars": 2.0,
    "Random 12 chars": 4.0,
    "Random 16 chars": 4.0,
    "5-word passphrase (27-word list)": 4.0
  },
  "word_year_symbol_pct_strong_before_cap": 65.5,
  "word_year_symbol_pct_strong_after_cap": 0.0,
  "streamlit_apptest_exceptions": 0
}
```

03-password-breach-checker/samples/strength_benchmark.json


```
{
  "PIN (digits only)": {
    "n": 200,
    "score_distribution": {
      "0": 1,
      "1": 199,
      "2": 0,
      "3": 0,
      "4": 0
    },
    "mean_score": 0.99,
    "pct_score_3_or_4": 0.0,
    "mean_naive_entropy_bits": 19.8,
    "mean_zxcvbn_log10_guesses": 5.4
  },
  "Keyboard walk + digits": {
    "n": 200,
    "score_distribution": {
      "0": 3,
      "1": 196,
      "2": 1,
      "3": 0,
      "4": 0
    },
    "mean_score": 0.99,
    "pct_score_3_or_4": 0.0,
    "mean_naive_entropy_bits": 35.6,
    "mean_zxcvbn_log10_guesses": 4.9
  },
  "Word + year + symbol": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 0,
      "2": 200,
      "3": 0,
      "4": 0
    },
    "mean_score": 2.0,
    "pct_score_3_or_4": 0.0,
    "mean_naive_entropy_bits": 73.1,
    "mean_zxcvbn_log10_guesses": 8.0
  },
  "l33t word + digits": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 182,
      "2": 18,
      "3": 0,
      "4": 0
    },
    "mean_score": 1.09,
    "pct_score_3_or_4": 0.0,
    "mean_naive_entropy_bits": 49.7,
    "mean_zxcvbn_log10_guesses": 5.7
  },
  "Random 8 chars": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 0,
      "2": 200,
      "3": 0,
      "4": 0
```

```
    },
    "mean_score": 2.0,
    "pct_score_3_or_4": 0.0,
    "mean_naive_entropy_bits": 50.0,
    "mean_zxcvbn_log10_guesses": 8.0
  },
  "Random 12 chars": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 0,
      "2": 0,
      "3": 0,
      "4": 200
    },
    "mean_score": 4.0,
    "pct_score_3_or_4": 100.0,
    "mean_naive_entropy_bits": 76.8,
    "mean_zxcvbn_log10_guesses": 12.0
  },
  "Random 16 chars": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 0,
      "2": 0,
      "3": 0,
      "4": 200
    },
    "mean_score": 4.0,
    "pct_score_3_or_4": 100.0,
    "mean_naive_entropy_bits": 103.2,
    "mean_zxcvbn_log10_guesses": 16.0
  },
  "5-word passphrase (27-word list)": {
    "n": 200,
    "score_distribution": {
      "0": 0,
      "1": 0,
      "2": 0,
      "3": 0,
      "4": 200
    },
    "mean_score": 4.0,
    "pct_score_3_or_4": 100.0,
    "mean_naive_entropy_bits": 202.7,
    "mean_zxcvbn_log10_guesses": 24.2
  }
}
```

03-password-breach-checker/scripts/benchmark_strength.py

```
"""Score SYNTHETIC passwords from common human patterns vs random generation
and report the score distribution per category. No real/leaked passwords are
bundled: every value is generated here from templates and a tiny word list.

    python scripts/benchmark_strength.py
Writes samples/strength_benchmark.json
"""
import json
import os
import random
import secrets
import string
import sys
from collections import Counter

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(ROOT, "src"))
from pwcheck.strength import analyze_strength  # noqa: E402

rng = random.Random(42)
WORDS = ["river", "garden", "monsoon", "tiger", "cricket", "mango", "silver", "rocket", "planet",
         "coffee", "window", "summer", "winter", "dragon", "falcon", "pickle", "velvet", "tractor",
         "harbor", "lantern", "meadow", "copper", "thunder", "orbit", "saffron", "bamboo", "canyon"]
SYMBOLS = "!@#$%&*?"


def word_year():
    return rng.choice(WORDS).capitalize() + str(rng.randint(1980, 2026)) + rng.choice(SYMBOLS)


def pin():
    return "".join(rng.choice(string.digits) for _ in range(rng.choice([4, 6, 8])))


def keyboard():
    row = rng.choice(["qwertyuiop", "asdfghjkl", "1234567890"])
    i = rng.randint(0, 3)
    return row[i:i + rng.randint(5, 7)] + str(rng.randint(0, 99))


def leet():
    w = rng.choice(WORDS)
    return w.replace("a", "@").replace("o", "0").replace("e", "3").capitalize() + str(rng.randint(1, 99))


def random_n(n):
    alphabet = string.ascii_letters + string.digits + SYMBOLS
    return lambda: "".join(secrets.choice(alphabet) for _ in range(n))


def passphrase():
    return " ".join(secrets.choice(WORDS) for _ in range(5))


CATEGORIES = {
    "PIN (digits only)": pin,
    "Keyboard walk + digits": keyboard,
    "Word + year + symbol": word_year,
    "l33t word + digits": leet,
    "Random 8 chars": random_n(8),
    "Random 12 chars": random_n(12),
    "Random 16 chars": random_n(16),
```

```
    "5-word passphrase (27-word list)": passphrase,
}
N = 200
results = {}
for name, gen in CATEGORIES.items():
    scores, ent, glog = Counter(), [], []
    for _ in range(N):
        r = analyze_strength(gen())
        scores[r.score] += 1
        ent.append(r.naive_entropy_bits)
        glog.append(r.zxcvbn_guesses_log10 or 0)
    results[name] = {
        "n": N, "score_distribution": {str(k): scores.get(k, 0) for k in range(5)},
        "mean_score": round(sum(k * v for k, v in scores.items()) / N, 2),
        "pct_score_3_or_4": round(100 * (scores[3] + scores[4]) / N, 1),
        "mean_naive_entropy_bits": round(sum(ent) / N, 1),
        "mean_zxcvbn_log10_guesses": round(sum(glog) / N, 1),
    }
with open(os.path.join(ROOT, "samples", "strength_benchmark.json"), "w") as f:
    json.dump(results, f, indent=2)
print(f"{'category':36s} {'mean':>5s} {'%3-4':>6s} {'naiveH':>7s} {'log10g':>7s}")
for k, v in results.items():
    print(f"{k:36s} {v['mean_score']:5.2f} {v['pct_score_3_or_4']:6.1f} "
          f"{v['mean_naive_entropy_bits']:7.1f} {v['mean_zxcvbn_log10_guesses']:7.1f}")

```

03-password-breach-checker/scripts/live_privacy_demo.py

```
"""Live demo against the real Pwned Passwords API using the PUBLIC test value
"password". Captures the exact prepared HTTP request (via a requests response
hook) to show what actually left the machine.

    python scripts/live_privacy_demo.py
"""
import json
import os
import sys

import requests

sys.path.insert(0, os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "src"))
from pwcheck import PwnedPasswordsClient, sha1_hex  # noqa: E402

DEMO = "password"  # public, famously breached; NOT a secret

captured = []
session = requests.Session()
session.hooks["response"].append(lambda r, *a, **k: captured.append(r))

full = sha1_hex(DEMO)
res = PwnedPasswordsClient(session=session, timeout=10).check(DEMO)
if not captured:
    print("Offline:", res.message)
    sys.exit(3)
r = captured[-1]
req = r.request
sent = f"{req.method} {req.url}\n" + "\n".join(f"{k}: {v}" for k, v in req.headers.items())

print("=== Exact request sent ===")
print(sent)
print(f"(request body: {req.body!r})")
print("\n=== Privacy assertions ===")
path_tail = req.url.rsplit("/", 1)[1]
checks = {
    "path ends in exactly 5 hex chars": len(path_tail) == 5 and all(c in "0123456789ABCDEF" for c in path_tail),
    "plaintext absent from request": DEMO not in sent.replace("pwnedpasswords", "") and not req.body,
    "35-char suffix absent from request": full[5:] not in sent,
    "full hash absent from request": full not in sent,
    "Add-Padding header present": req.headers.get("Add-Padding") == "true",
}
for k, v in checks.items():
    print(f"[{'PASS' if v else 'FAIL'}] {k}")
print("\n=== Response (matched locally) ===")
print(f"HTTP {r.status_code}, {len(r.content):,} bytes, {res.bucket_size:,} real suffixes + "
      f"{res.padding_entries} padding rows for prefix {res.prefix}")
print(json.dumps(res.to_dict(), indent=2))
sys.exit(0 if all(checks.values()) and res.pwned else 1)

```

03-password-breach-checker/scripts/measure_anonymity.py

```python
"""Measure the k-anonymity set size and latency on the live API using RANDOM
throwaway strings (generated here, never real passwords).

    python scripts/measure_anonymity.py --n 25
Writes samples/anonymity_measurement.json
"""
import argparse
import json
import os
import secrets
import statistics
import sys
import time

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(ROOT, "src"))
from pwcheck import PwnedPasswordsClient  # noqa: E402

ap = argparse.ArgumentParser()
ap.add_argument("--n", type=int, default=25)
args = ap.parse_args()

client = PwnedPasswordsClient(timeout=10)
rows = []
for _ in range(args.n):
    throwaway = secrets.token_urlsafe(24)   # random, never stored
    r = client.check(throwaway)
    rows.append({"status": r.status, "bucket_size": r.bucket_size,
                 "padding_entries": r.padding_entries, "elapsed_ms": round(r.elapsed_ms, 1)})
    time.sleep(0.2)  # be polite
ok = [x for x in rows if x["status"] in ("found", "not_found")]
summary = {
    "lookups": len(rows), "succeeded": len(ok),
    "found_in_breaches": sum(x["status"] == "found" for x in ok),
    "bucket_size_min": min(x["bucket_size"] for x in ok) if ok else None,
    "bucket_size_median": statistics.median(x["bucket_size"] for x in ok) if ok else None,
    "bucket_size_max": max(x["bucket_size"] for x in ok) if ok else None,
    "padding_rows_median": statistics.median(x["padding_entries"] for x in ok) if ok else None,
    "latency_ms_median": statistics.median(x["elapsed_ms"] for x in ok) if ok else None,
    "latency_ms_max": max(x["elapsed_ms"] for x in ok) if ok else None,
    "measured_at_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
}
os.makedirs(os.path.join(ROOT, "samples"), exist_ok=True)
with open(os.path.join(ROOT, "samples", "anonymity_measurement.json"), "w") as f:
    json.dump({"summary": summary, "lookups": rows}, f, indent=2)
print(json.dumps(summary, indent=2))

```